

How to install/run via pip and uv

The project was designed to run in a clean, reproducible environment using virtual environments and dependency management tools.

To install using pip under module_5:

1. Create and activate a virtual environment:
 - a. `python -m venv .venv`
 - b. `.venv\Scripts\activate`
2. Install dependencies:
 - a. `pip install -r requirements.txt`
3. Set the database environment variable:
 - a. `$env:DATABASE_URL="postgresql://app_user:<PASSWORD>@localhost:5432/module_3db"`
4. Run the application:
 - a. `python -m src.app`
5. To install using uv:
 - a. `uv venv`
 - b. `uv pip sync requirements.txt`

Dependency Graph Explanation

The dependency graph generated using “pydeps” shows how the main application module depends on supporting libraries and internal modules.

The Flask framework is the central dependency, providing routing and request handling. The psycopg library is used for communication with the PostgreSQL database. The “sql_utils” module handles SQL normalization and LIMIT enforcement. Testing and security tools such as pytest, pylint, and coverage are also known as development dependencies.

The graph illustrates a modular structure where business logic, database access, and utilities are separated. This improves maintainability and security by isolating responsibilities. External libraries are used only when necessary, reducing attack surface and complexity.

SQL Injection Defenses

All database queries were refactored to prevent SQL injection vulnerabilities. Dynamic SQL construction using string concatenation, f-strings, or format methods was removed.

Instead, the application uses psycopg’s parametrized queries and SQL composition utilities. User provided values are passed through parameter binding rather than inserted directly into SQL strings. Dynamic identifiers are safely quoted using “psycopg.sql.identifier”.

Additionally, all queries are normalized and required to include a LIMIT clause using a custom enforcement function. This prevents accidental full-table scans and limits potential data exposure.

These changes ensure that malicious input cannot alter query structure or execute unintended commands.

```

    with get_conn() as conn:
        with conn.cursor() as cur:
            safe_sql = _ensure_limit(sql)
            cur.execute(safe_sql, params or {})
            rows = cur.fetchall()
            headers = [d.name for d in cur.description] if cur.description else []

    return headers, rows

normalized = sql_utils.normalize_sql(sql)
return normalized if sql_utils.has_limit(normalized) else f"{normalized} LIMIT 1"

```

Least-Privilege Database Configuration

We use this:

Least-Privilege Database Configuration

The application uses a dedicated database user with restricted privileges instead of a superuser account. The app_user account was configured with the following permissions:

- Usage on schema public
- SELECT, INSERT, UPDATE, and DELETE on the applicants table
- SELECT and USAGE on required sequences

The user does not have DROP, ALTER, or ownership privileges.

Here is my example SQL configuration:

```

GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE applicants TO app_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO app_user;

```

Default privileges were also configured to ensure future tables inherit appropriate permissions. This setup follows the principle of least privilege and limits damage in the event of credential compromise.

Security and Testing proof:

```

(.venv) PS D:\School\JHU\Modern Softwaree Concepts in Python\code\jhu_software_concepts\module_5> python -m pylint src --fail-under=10

-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

```

```
(.venv) PS D:\School\JHU\Modern Softwaree Concepts in Python\code\jhu_software_concepts\module_5> python -m pytest -q
.....5.....
----- tests coverage -----
coverage: platform win32, python 3.13.3-final-0 -----

Name           Stmts  Miss  Cover   Missing
-----
src\__init__.py    0     0   100%
src\app.py        95     0   100%
src\clean.py      59     0   100%
src\run.py         9     0   100%
src\sql_utils.py  19     0   100%
-----
TOTAL             182     0   100%
Required test coverage of 100% reached. Total coverage: 100.00%
```

```
(.venv) PS D:\School\JHU\Modern Softwaree Concepts in Python\code\jhu_software_concepts\module_5> snyk test

Testing D:\School\JHU\Modern Softwaree Concepts in Python\code\jhu_software_concepts\module_5...

Organization:    addresscrp
Package manager: pip
Target file:     requirements.txt
Project name:    module_5
Open source:     no
Project path:    D:\School\JHU\Modern Softwaree Concepts in Python\code\jhu_software_concepts\module_5
Licenses:        enabled

✓ Tested 52 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```